DOBI: Mathematical Formulation and Parameter Analysis

Xiuyuan Lu

1 Introduction

The Dombi-Bonferroni (DOBI) method is a novel multi-criteria decision making (MCDM) technique that combines Dombi operations with Bonferroni mean operators. This document presents its mathematical formulation and analyzes the effects of its key parameters.

2 Mathematical Formulation

2.1 Data Normalization

The method begins with normalized decision matrix X^* where:

$$x_{ij}^* = \begin{cases} \dfrac{x_{ij}}{\max(x_j)} & \text{(benefit criterion)} \\ \dfrac{\min(x_j)}{x_{ij}} + \dfrac{\max(x_j)}{\max(x_j)} + \dfrac{\min(x_j)}{\max(x_j)} & \text{(cost criterion)} \end{cases} \tag{1}$$

2.2 Additive Normalization (f-values)

$$f_{ij} = \frac{x_{ij}^*}{\sum_{j=1}^{n} x_{ij}^*} \tag{2}$$

2.3 Dombi-Bonferroni Functions

2.3.1 Weighted Averaging Function (Z1)

$$Z1_i = \frac{\sum_{j=1}^{n} x_{ij}^*}{1 + \left[\dfrac{\sum_{j=1}^{n} \sum_{\substack{k=1 \\ k \neq j}}^{n} \frac{w_j w_k}{1-w_j} \left(\psi_1 \left(\frac{1-f_{ij}}{f_{ij}} \right)^{\zeta} + \psi_2 \left(\frac{1-f_{ik}}{f_{ik}} \right)^{\zeta} \right)}{\psi_1 + \psi_2} \right]^{1/\zeta}} \tag{3}$$

2.3.2 Weighted Geometric Function (Z2)

$$Z2_i = \frac{\sum_{j=1}^{n} x_{ij}^*}{1 + \left[\dfrac{\sum_{j=1}^{n} \sum_{\substack{k=1 \\ k \neq j}}^{n} \frac{w_j w_k}{1-w_j} \left(\psi_1 \left(\frac{f_{ij}}{1-f_{ij}} \right)^{\zeta} + \psi_2 \left(\frac{f_{ik}}{1-f_{ik}} \right)^{\zeta} \right)}{\psi_1 + \psi_2} \right]^{1/\zeta}} \tag{4}$$

2.4 Integrated DOBI Value

$$\Re_i = \frac{Z1_i + Z2_i}{1 + \left[0.5 \left(\frac{1-Z1_i}{Z1_i} \right)^{\delta} + 0.5 \left(\frac{1-Z2_i}{Z2_i} \right)^{\delta} \right]^{1/\delta}} \tag{5}$$

2.5 Ranking

$$\text{Rank}_i = \text{rank}(\Re_i, \text{ascending=False}) \tag{6}$$

Table 1: DOBI Parameters and Their Effects

Parameter	Role	Impact
ψ_1, ψ_2	Bonferroni parameters	Control inter-criteria compensation
ζ	Dombi operator parameter	Adjusts aggregation behavior
δ	Integration parameter	Balances Z1 and Z2 contributions

3 Parameter Analysis

3.1 Effects of Parameter Settings

3.1.1 ψ_1 and ψ_2 (Bonferroni Parameters)

- **Symmetry**: When $\psi_1 = \psi_2$, equal weight to all interactions
- **Compensation**: Higher values increase compensation effects
- **Robustness**: Asymmetric values ($\psi_1 \neq \psi_2$) increase sensitivity to specific criteria

3.1.2 ζ (Dombi Operator Parameter)

$$\lim_{\zeta \to 0} \to \text{maximum-like operation}, \quad \lim_{\zeta \to \infty} \to \text{minimum-like operation} \tag{7}$$

- Small ζ: More sensitive to high values
- Large ζ: More sensitive to low values
- $\zeta = 1$: Linear compensation

3.1.3 δ (Integration Parameter)

- $\delta = 1$: Equal treatment of Z1 and Z2
- $\delta > 1$: Emphasizes larger values
- $\delta < 1$: Reduces impact of extreme values

4 Recommended Parameter Settings

Table 2: Parameter Setting Scenarios

Scenario	ψ_1, ψ_2	ζ	δ	Application
Default	1,1	1	1	General purpose
Robust	1,1	2	0.5	Noisy data
Dominance	2,2	0.5	2	Key criteria emphasis

5 Implementation Notes

The Python implementation features:

- Direct matrix operations for efficiency
- Flexible parameter configuration
- Batch processing for temporal data

For sensitivity analysis, we recommend:

```
param_combinations = [
    (1, 1, 1, 1),  # Baseline
    (1, 2, 1, 1),  # Asymmetric psi
    (1, 1, 2, 1),  # Increased zeta
    (1, 1, 1, 2)   # Increased delta
]
```

6 Appendix

Here is the source code for DOBI implementation in Python:

```
import numpy as np
import pandas as pd

years = [2010, 2013, 2016, 2019, 2022]
indicators = [f'X{i}' for i in range(1, 9)]
column_names = ['State'] + indicators
negative_indicators = ['X8']
states = (pd.read_excel('../raw_data/2022data.xlsx'))['State'].to_list()
psi1, psi2, zeta, delta = 1, 1, 1, 1

def read_weights(year):
    weights = pd.read_excel(f'../processed_data/{year}_weights.xlsx')
    weights.columns = ['method'] + indicators
    weight = weights[weights['method'] == 'IDOCRIW'][indicators].to_numpy()[0]  #      [0]1 D
    return weight

def DOBI(year):
    # read the normalized data
    data = pd.read_excel(f'../processed_data/{year}_vector_normalized.xlsx')[indicators].to_numpy()

    # read weights
    weights = read_weights(year)

    # 2. calculate value of f
    row_sums = np.sum(data, axis=1, keepdims=True)
    f_values = data / row_sums

    # 3. DOBI function
    n_samples, n_features = data.shape
    Z1 = np.zeros(n_samples)
    Z2 = np.zeros(n_samples)

    for i in range(n_samples):
        # Z1
        numerator = np.sum(data[i, :])
        denominator_terms = []
        for j in range(n_features):
            for k in range(n_features):
                if j != k:
                    wj = weights[j]
                    wk = weights[k]
                    term1 = psi1 * ((1 - f_values[i, j]) / f_values[i, j]) ** zeta
                    term2 = psi2 * ((1 - f_values[i, k]) / f_values[i, k]) ** zeta
                    denominator_terms.append((wj * wk / (1 - wj)) * (term1 + term2))
        Z1[i] = numerator / (1 + (np.sum(denominator_terms) / (psi1 + psi2)) ** (1 / zeta))

        # Z2
        denominator_terms = []
        for j in range(n_features):
            for k in range(n_features):
                if j != k:
                    wj = weights[j]
                    wk = weights[k]
                    term1 = psi1 * (f_values[i, j] / (1 - f_values[i, j])) ** zeta
                    term2 = psi2 * (f_values[i, k] / (1 - f_values[i, k])) ** zeta
                    denominator_terms.append((wj * wk / (1 - wj)) * (term1 + term2))
        Z2[i] = numerator / (1 + (np.sum(denominator_terms) / (psi1 + psi2)) ** (1 / zeta))

    # 4. integrated values
    term1 = 0.5 * ((1 - Z1) / Z1) ** delta
    term2 = 0.5 * ((1 - Z2) / Z2) ** delta
    integrated_values = (Z1 + Z2) / (1 + (term1 + term2) ** (1 / delta))
    # Rank the alternatives
```

```
    result = pd.DataFrame({
        'State': states,
        'DOBI': integrated_values
    })
    result['Ranking'] = result['DOBI'].rank(ascending=False, method='min').astype(int)
    print(f'{year}\n', result)
    return result

for year in years:
    DOBI(year).to_excel(f'../processed_data/{year}_DOBI.xlsx', index=False)
```